

IOT PROJECT REPORT

Physical Notification Bell

A complete IoT solution — Arduino · Gateway · MQTT · Node-RED · Dashboard

Students	Adam HADOUTI · Aya EL MOUDEN
Group	TP C
Professors	Ouassim KARRAKCHOU · Nouhaila FRAIHI
Institution	UIR — International University of Rabat
Academic Year	2025–2026

1. Project Overview

1.1 Concept

The Physical Notification Bell is an IoT device that bridges the digital and physical worlds. A user types a message on a web dashboard; one second later the physical device on a desk buzzes, flashes, and displays the message on an LCD screen. The recipient presses a button to acknowledge — sending a read-receipt back to the dashboard in real time.

This demonstrates a complete bidirectional IoT loop: cloud sends to device, device responds to cloud.

1.2 Real-World Use Case

Hospital bedside communication — a nurse or caregiver sends a message remotely to a patient's bedside device: "Lunch is ready" or "Doctor arriving in 5 min". The patient presses the acknowledgement button. If no response within 10 minutes, the dashboard raises an alert. This exact architecture is used commercially by companies such as Vocera for patient communication systems.

1.3 Why IoT?

- Physical device reacts to cloud events in real time
- Confirmation flows back from device to cloud (bidirectional MQTT)
- All logic is distributed: Arduino handles hardware, Node-RED handles business logic
- Scalable: one dashboard can manage multiple devices simultaneously

2. System Architecture

The system is organised into four distinct layers communicating exclusively via MQTT over a local broker:

Layer	Technology	Role
Device	Arduino Uno + components	Receives messages, alerts user, sends acknowledgement
Gateway	Python 3 — Mac M1	Bridges USB serial ↔ MQTT broker
Broker	Mosquitto (localhost:1883)	Routes all MQTT messages between layers
Application	Node-RED + Dashboard UI	Business logic, message history, analytics

2.1 Communication Flow

Dashboard → publishes JSON to bell/message → Mosquitto routes to Gateway → Gateway forwards over USB serial → Arduino displays message + alerts user → User presses button → Arduino sends ACK:timestamp over serial → Gateway publishes to bell/ack → Dashboard shows Read status.

2.2 MQTT Topics

Topic	Direction	Payload	Purpose
bell/message	Dashboard → Device	{"message":"...", "priority":false}	Send a message to the device
bell/ack	Device → Dashboard	{"event":"acknowledged", "timestamp":"..."}	Read receipt from device
bell/clear	Dashboard → Device	"CLEAR"	Clear LCD remotely

3. Hardware Components

3.1 Sensors

Sensor	Arduino Pin	Role
Push button	Pin 7	User presses to acknowledge message — sends read-receipt to dashboard
RTC module (DS3231)	A4 / A5 (I2C)	Timestamps message delivery and acknowledgement events

3.2 Actuators

Actuator	Arduino Pin	Role
LCD screen (16×2 I2C)	A4 / A5 (I2C)	Displays incoming message; scrolls text longer than 16 chars
Buzzer module	Pin 8	Plays 3-beep tone on normal message; rapid pattern for priority
LED	Pin 6	Flashes on arrival; blinks while unread; escalates after 30s

3.3 Hardware Photo

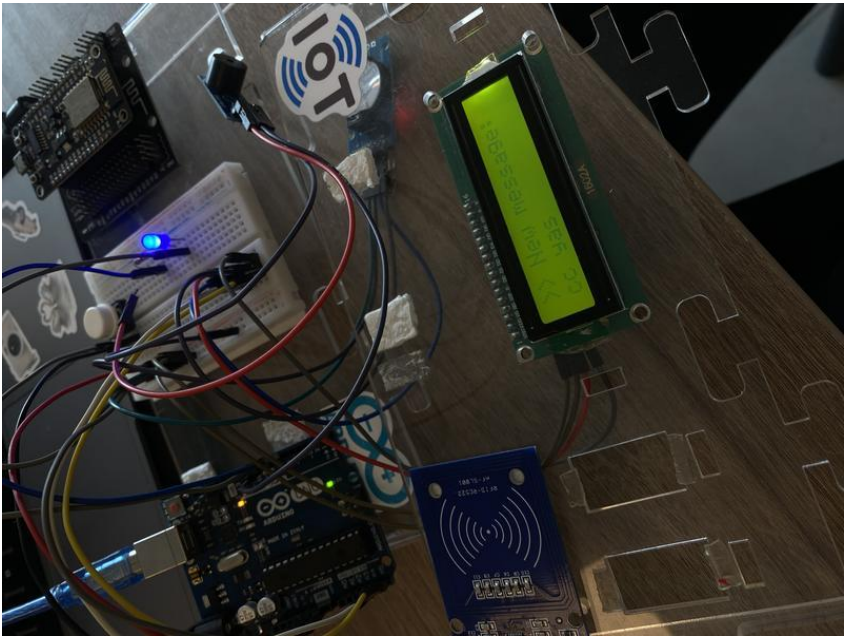


Fig 1 — Assembled hardware: Arduino Uno, breadboard, button, LED, LCD, RTC module

3.4 Wiring Summary

Component	Arduino Pin
LCD SDA	A4
LCD SCL	A5
LCD VCC	5V
LCD GND	GND
RTC SDA	A4 (shared I2C bus)
RTC SCL	A5 (shared I2C bus)
Buzzer VCC	5V
Buzzer GND	GND
Buzzer I/O	Pin 8
Button	Pin 7 → GND (INPUT_PULLUP)
LED +	Pin 6 → 220Ω resistor → GND

4. Software Implementation

4.1 Arduino Code

The Arduino firmware (C++) handles all hardware interactions. On startup it initialises the LCD, buzzer, LED, button and RTC. The main loop continuously monitors the serial port for incoming messages from the gateway and the button state for acknowledgements.

- MSG: prefix → normal message — plays 3-beep tone, flashes LED 3 times, displays text on LCD
- PRIORITY: prefix → urgent message — rapid 5-beep alarm, LED flashes 6 times
- CLEAR command → clears LCD and resets state
- Button press → sends ACK:<unix_timestamp> over serial, plays confirmation tone
- LED escalation → blink rate doubles after 30s of unread message

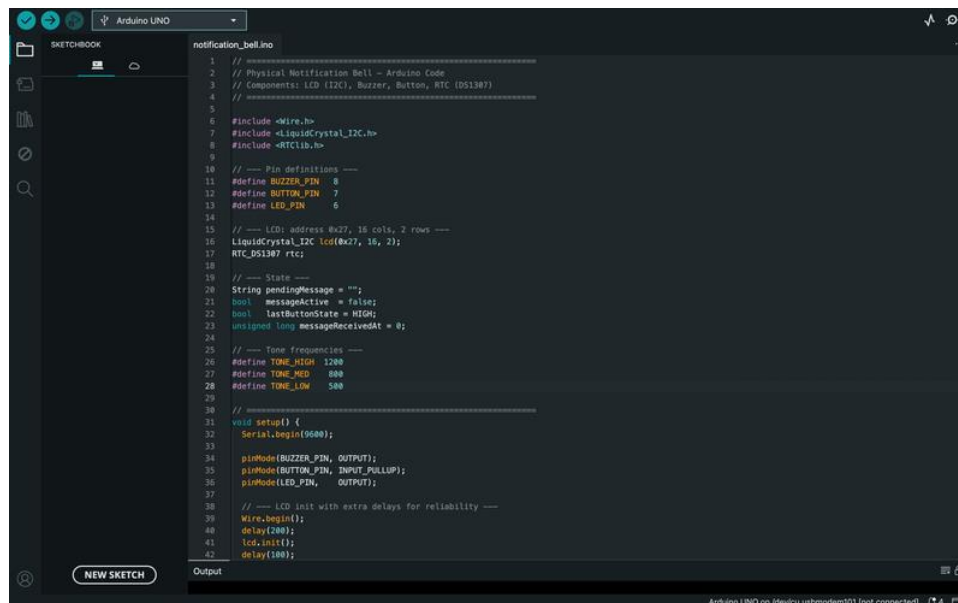


Fig 2 — Arduino code: setup function with LCD initialisation and pin configuration

4.2 Gateway Script (Python)

The Python gateway (gateway.py) runs on the Mac M1 and serves as the bridge between the MQTT broker and the Arduino. It subscribes to bell/message and bell/clear topics, forwards commands to Arduino over USB serial, reads acknowledgements from serial, and publishes them back to MQTT.

- --port flag specifies the Arduino USB serial port (e.g. /dev/tty.usbmodem101)
- --mock flag enables offline testing mode using keyboard input instead of serial
- Threading: serial reading runs in a background thread; MQTT loop runs in main thread
- All MQTT topics and serial baud rate (9600) are configurable at the top of the file

```

13 import argparse
14 import json
15 import sys
16 import time
17 import threading
18 from datetime import datetime
19
20 import paho.mqtt.client as mqtt
21
22 # --- Config ---
23 MQTT_BROKER = "localhost"
24 MQTT_PORT = 1883
25 MQTT_USER = ""
26 MQTT_PASSWORD = ""
27
28 # MQTT Topics
29 TOPIC_SEND = "bell/message" # dashboard - gateway - arduino
30 TOPIC_ACK = "bell/ack" # arduino - gateway - dashboard
31 TOPIC_STATUS = "bell/status" # arduino - gateway - dashboard
32 TOPIC_CLEAR = "bell/clear" # dashboard - arduino (clear LCD)
33
34 # --- Globals ---
35 serial_conn = None
36 mock_mode = False
37
38 # MQTT Callbacks
39
40 def on_connect(client, userdata, flags, rc):
41     if rc == 0:
42         print(f"[MQTT] Connected to broker at {MQTT_BROKER}:{MQTT_PORT}")
43         client.subscribe(TOPIC_SEND)
44         client.subscribe(TOPIC_CLEAR)
45         print(f"[MQTT] Subscribed to: {TOPIC_SEND}, {TOPIC_CLEAR}")
46     else:
47         print(f"[MQTT] Connection failed with code {rc}")
48
49 def on_message(client, userdata, msg):
50     topic = msg.topic
51     payload = msg.payload.decode("utf-8").strip()
52     print(f"[MQTT] Received on '{topic}': {payload}")
53
54 if __name__ == '__main__':
55     client = mqtt.Client()
56     client.on_connect = on_connect
57     client.on_message = on_message
58     client.connect(MQTT_BROKER, MQTT_PORT, 60)
59     client.loop_start()
60     while True:
61         # Main loop logic
62         pass
63     client.loop_stop()

```

Fig 3 — Gateway Python script: MQTT configuration and topic definitions

4.3 Node-RED Flow

Node-RED implements all business logic. The flow handles message construction, MQTT publishing, acknowledgement processing, response-time calculation, and dashboard updates. No business logic resides in the Arduino or gateway — they are pure I/O layers.

- Message Input node captures text from the dashboard text field
- Priority Alert switch stores the priority flag in flow context
- Build and send message function assembles the JSON payload and tracks session stats
- Handle acknowledgement function computes response time and updates all dashboard widgets
- All MQTT connections use the local Mosquitto broker on localhost:1883

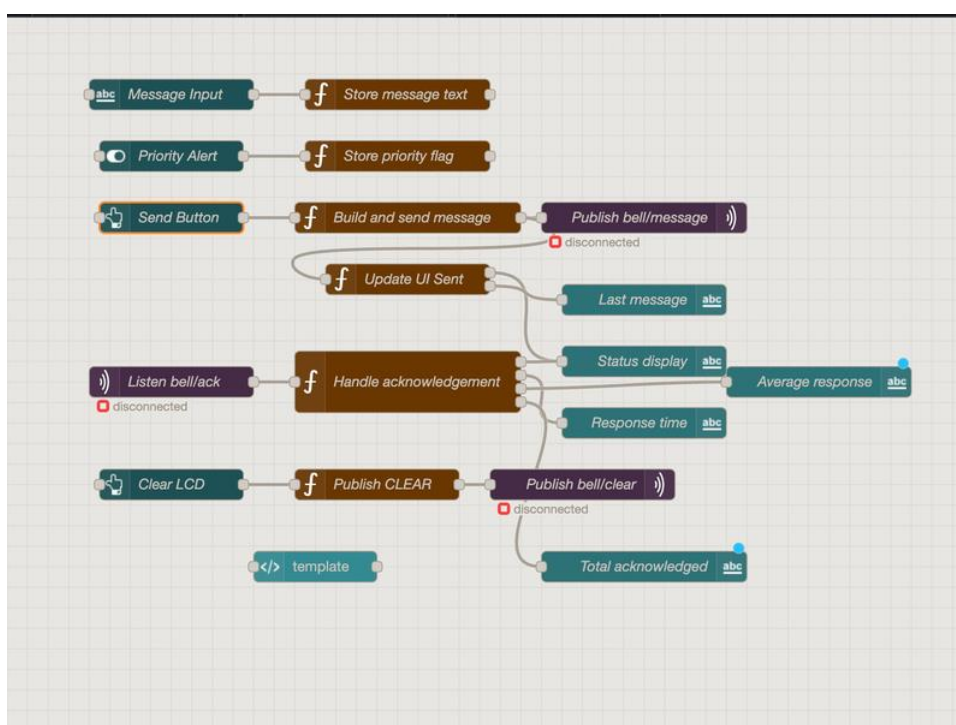


Fig 4 — Node-RED flow: complete message send and acknowledgement pipeline

5. Dashboard

5.1 Features

Feature	Description
Message composer	Text input (max 32 chars) + Send button
Priority toggle	Marks message as urgent — different buzzer pattern on device
Delivery status	Live indicator: Sent → Delivered → Read with timestamps
Response time	Time in seconds from send to acknowledgement
Session stats	Sent count, read count, average response time
Event log	Timestamped log of all sent messages and acknowledgements
Clear LCD	Sends CLEAR command to device remotely via MQTT

5.2 Dashboard Screenshot

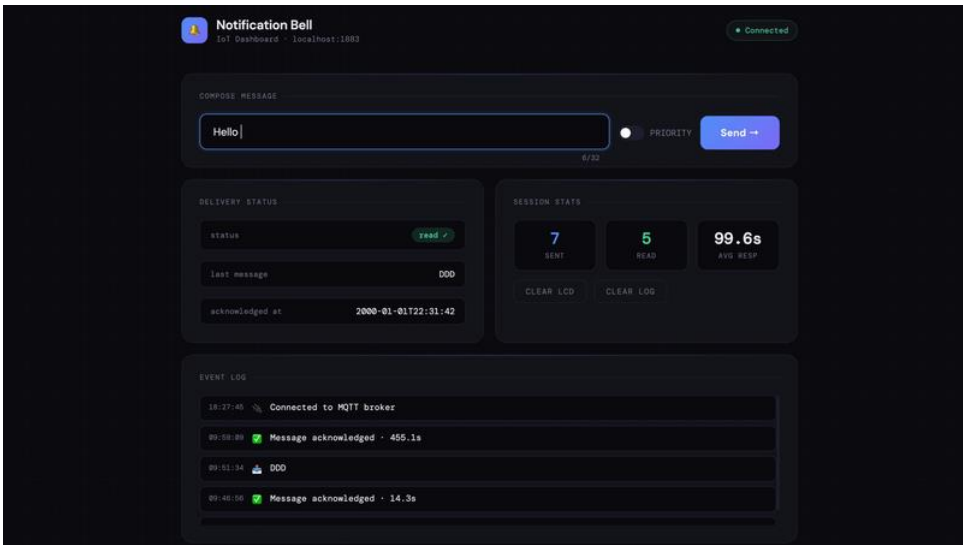


Fig 5 — Custom HTML dashboard: Connected status, message composer, delivery status, session stats and event log

The dashboard connects to Mosquitto via WebSocket (port 9001) and is served locally via Python HTTP server. It shows real-time delivery status, response time analytics, and a full event log with timestamps.

6. Algorithm

6.1 Cloud → Device (Message Delivery)

- User types message and clicks Send on dashboard
- Node-RED publishes JSON payload to MQTT topic bell/message
- Gateway receives via MQTT and forwards MSG:<text> to Arduino over USB serial
- Arduino plays buzzer tone, flashes LED, displays message on LCD

- RTC records delivery timestamp

6.2 Device → Cloud (Acknowledgement)

- User presses physical button on device
- Arduino reads RTC timestamp and sends ACK:<unix_timestamp> over serial
- Gateway publishes JSON to MQTT topic bell/ack
- Node-RED computes response time = ack_time – send_time
- Dashboard updates status from Delivered to Read ✓

6.3 Escalation Logic

- Unread for 30+ seconds → LED blink rate doubles
- Dashboard status remains Delivered until button is pressed
- Priority messages trigger a distinct rapid buzzer pattern

8. Conclusion

This project successfully implements a complete IoT solution demonstrating all four required layers: Arduino firmware, Python gateway, Node-RED business logic, and a custom web dashboard. The Physical Notification Bell goes beyond a simple sensor-to-actuator mapping by implementing a full bidirectional communication loop with delivery confirmation, escalation logic, and real-time analytics.

The most technically significant achievement is the bidirectional MQTT architecture: the dashboard sends commands down to the device, and the device sends confirmations back up to the dashboard. This pattern — used in real enterprise IoT systems — ensures that the human action was actually taken, not just that the alert was sent.

All non-hardware components (gateway, Node-RED flow, dashboard) were developed and tested offline before lab sessions, minimising hardware debugging time and allowing both sessions to focus on integration and polish.

Adam HADOUTI · Aya EL MOUDEN · TP C · 2025–2026